Implementation of a CAN/CAN-FD Bus Transmitter in VHDL

Mads Richardt (s224948)

February 28, 2026

Contents

1 Design and Architecture

1.1 System Overview

A complete CAN node is composed of a Transmit (TX) path and a Receive (RX) path coordinated by shared fault-confinement and physical-interface control. The paths operate in parallel at runtime, but they are not symmetric in behavior or responsibility. The focus of this thesis is the complete node architecture and its standards-traceable interfaces. The TX path is responsible for frame submission, arbitration participation, and bitstream generation toward the bus, while the RX path is responsible for bus observation, frame reconstruction, and delivery/notification toward the user layer. As shown in Figure 1, this full-node decomposition spans LLC, MAC, PCS, FCE, and PMA boundaries.

1.1.1 LLC Frame Format

The current `LLC Frame` format is depicted in Figure 2 with the ID byte format depicted in Table 1. The revised `LLC Frame` supporting FD is depicted in Figure 3. The revised format adds a 3-bit `FMT` (ISO 6.4.3) field to the DLC byte, expands the data field to 64 bytes, and repurposes reserved bits in the last byte for BRS and ESI flags. The `FMT` encodes the supported frame formats as '`000`' = CB, '`100`' = CE, '`010`' = FB, '`110`' = FE. Accordingly, for the frame content to be self-consistent the IDE bit must be set to `0` for `FMT` = CB/FB and `1` for `FMT` = CE/FE. The revised format is designed to be backward compatible: an implementation that only supports Classic frames can simply ignore the `FMT` bits and treat all frames as Classic, while an implementation that supports FD can use the `FMT` field to distinguish frame types without affecting the

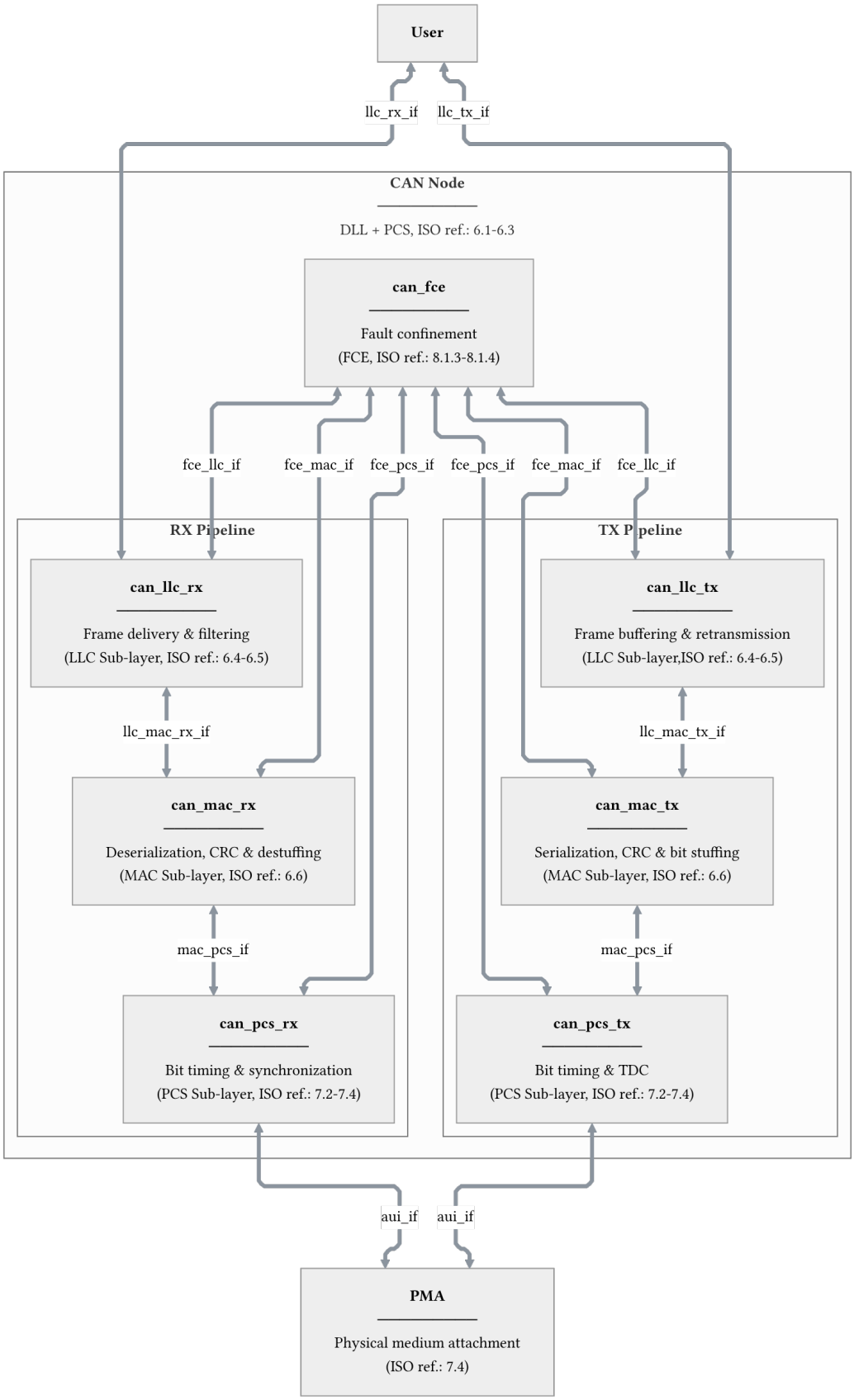

Figure 1: CAN node decomposition across LLC, MAC, PCS, FCE, and PMA boundaries. Interface definitions are provided for llc_tx_if (Table 2), llc_rx_if (Table 3), llc_mac_tx_if (Table 4), llc_mac_rx_if (Table 5), mac_pcs_if (Table 6), aui_if (Table 7), fce_llc_if (Table 8), fce_mac_if (Table 9), and fce_pcs_if (Table 10).

existing ID and data field structure.

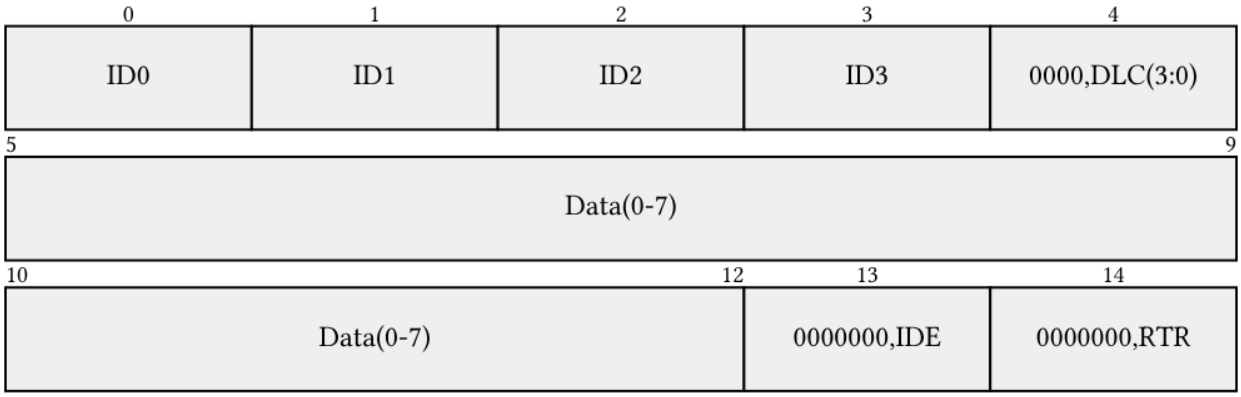

Figure 2: Current 15 byte LLC Frame format.

Table 1: Four ID byts of the current `LLC frame` format.

Format	Byte ID0	Byte ID1	Byte ID2	Byte ID3
Basic	'00000000'	'00000000'	'00000' & 'ID(10-8)'	'ID(7-0)'
Extended	'000' & 'ID(28:24)'	'ID(23-16)'	'ID(15-8)'	'ID(7-0)'

1.2 Interface Definition Tables

The following tables define the interface bundles shown in Figure 1. ISO references are to ISO 11898-1:2024 clauses and tables.

Detailed interface definitions (normative source mapping):

Figure 3: Maximum length revised LLC Frame format with FD support.

Table 2: Interface definition for `llc_tx_if`. Implements `L_Data.Request` as an Avalon-ST byte stream. `L_Data.AbortRequest` is included for complete ISO service coverage but is not yet implemented.

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
6.4.5.5.2	`L_Data.Request`	`LLC Frame`, `Handle`	Submit LLC frame for transmission	`User -> LLC`	`llc_tx_if.data` (`byte_t`), `llc_tx_if.valid` (`std_logic`), `llc_tx_if.sop` (`std_logic`)	`valid` high with byte on data; `sop` asserted on first byte
6.4.5.5.2	`L_Data.Request`	`LLC Frame`, `Handle`	Submit LLC frame for transmission	`User -> LLC`	`llc_tx_if.data` (`byte_t`), `llc_tx_if.valid` (`std_logic`)	`valid` high; `sop` and `eop` deasserted on intermediate bytes
6.4.5.5.2	`L_Data.Request`	`LLC Frame`, `Handle`	Submit LLC frame for transmission	`User -> LLC`	`llc_tx_if.data` (`byte_t`), `llc_tx_if.valid` (`std_logic`), `llc_tx_if.eop` (`std_logic`)	`valid` high with byte on data; `eop` asserted on last byte
6.4.5.5.2	`L_Data.Request`		Flow control	`LLC -> User`	`llc_tx_if.ready` (`std_logic`)	Asserted by LLC when able to consume next byte
6.4.5.5.3	`L_Data.AbortRequest`	`Handle`	Abort pending frame transfer	`User -> LLC`	`llc_tx_if.abort_request` (`std_logic`)	Pulse when user wants to cancel an in-progress transfer

Table 3: Interface definition for `llc_rx_if`. Implements `L_Data.Indication` as an Avalon-ST byte stream. `Timestamp` is included for complete ISO service coverage but is not yet implemented.

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
6.4.5.5.5	`L_Data.Indication`	`LLC Frame`, `Timestamp`	Deliver received LLC frame to user	`LLC -> User`	`llc_rx_if.data` (`byte_t`), `llc_rx_if.valid` (`std_logic`), `llc_rx_if.sop` (`std_logic`)	`valid` high with byte on data; `sop` asserted on first byte

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
6.4.5.5.5	`L_Data.Indication`	`LLC Frame`, `Timestamp`	Deliver received LLC frame to user	`LLC -> User`	`llc_rx_if.data` (`byte_t`), `llc_rx_if.valid` (`std_logic`)	`valid` high; `sop` and `eop` deasserted on intermediate bytes
6.4.5.5.5	`L_Data.Indication`	`LLC Frame`, `Timestamp`	Deliver received LLC frame to user	`LLC -> User`	`llc_rx_if.data` (`byte_t`), `llc_rx_if.valid` (`std_logic`), `llc_rx_if.eop` (`std_logic`)	`valid` high with byte on data; `eop` asserted on last byte
6.4.5.5.5	`L_Data.Indication`		Flow control	`User -> LLC`	`llc_rx_if.ready` (`std_logic`)	Asserted by user when able to consume next byte
6.4.5.5.4	`L_Data.Confirm`	`Transfer_Status`, `Timestamp`, `Handle`	Report outcome of prior `L_Data.Request`	`LLC -> User`	`llc_rx_if.transfer_status` (`transfer_status_t`)	Issued on frame completion, loss, or error

Table 4: Interface definition for `llc_mac_tx_if`. Frame length is self-describing from the DLC config bytes; the MAC serializer does not consume `eop`.

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
6.3, 6.6.4.2	`DLL SDU`	`LLC Frame`	Transfer LLC frame to MAC for serialization	`LLC -> MAC`	`llc_mac_tx_if.data` (`byte_t`), `llc_mac_tx_if.valid` (`std_logic`), `llc_mac_tx_if.sop` (`std_logic`)	`valid` high with byte on data; `sop` marks first byte of new frame
6.3, 6.6.4.2	`DLL SDU`		Flow control	`MAC -> LLC`	`llc_mac_tx_if.ready` (`std_logic`)	Asserted by MAC serializer when able to consume next byte
6.6.4.2	Interface control information	`Transfer_Status`	Report frame transmission outcome	`MAC -> LLC`	`llc_mac_tx_if.transfer_status` (`transfer_status_t`)	Updated by MAC on completion, loss, or error

Table 5: Interface definition for `llc_mac_rx_if`.

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
6.6.4.3, 6.6.9	`DLL SDU`	Reconstructed `LLC Frame`	Transfer reconstructed LLC frame to LLC	`MAC -> LLC`		Issued when MAC has reconstructed a frame from the received bitstream
6.6.3	Time reference notification	`SOF/frame-valid indication`	Provide timing reference for timestamping	`MAC -> LLC`		Generated for transmitted/received DF/RF

Table 6: Interface definition for `mac_pcs_if`. The `D_Transmit` status is encoded directly in `mac_pcs_if.data.bit_name` rather than as a separate signal - the PCS derives data-phase entry and exit from the protocol-semantic bit name, eliminating the need for a redundant boolean flag.

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
7.2.1, 7.2.2	`PCS_Data.Request`	`Output_Unit`	Present next bit for transmission	`MAC -> PCS`	`mac_pcs_if.data` (`mac_frame_bit_t`)	MAC holds bit stable; PCS samples `data` at each bit boundary autonomously
7.2.1, 7.2.5	`PCS_Status.Transmitter`	`D_Transmit`	Indicate FD data-phase interval to PCS	`MAC -> PCS`	`mac_pcs_if.data.bit_name` (`mac_frame_bit_name_t`)	PCS enters data phase at `fdf_bit` SP, exits at `crc_delimiter_bit` or error flag boundary
7.2.1, 7.2.3	`PCS_Data.Indicate`	`Input_Unit`	Indicate bus polarity at sample point	`PCS -> MAC`	`mac_pcs_if.bus_polarity` (`polarity_t`), `mac_pcs_if.sample_strobe` (`std_logic`), `mac_pcs_if.strobe_type` (`strobe_type_t`), `mac_pcs_if.fifo_index` (`integer`)	`sample_strobe` pulses once per SP/SSP; `strobe_type` distinguishes SP from SSP for TDC

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
7.2.1, 7.2.6	`PCS_Status.Receiver`	`D_Receive`	Indicate FD data-phase interval to PCS	`MAC -> PCS`	not yet implemented	Asserted during FD data-phase reception

Table 7: Interface definition for `aui_if`.

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
7.4.2.1	`output symbol`	`Dominant/recessive symbol`	Drive physical output symbol	`PCS -> PMA`	`aui_if.tx` (`std_logic`)	Updated on each `Output_Unit` request
7.4.2.2, 8.1.3.4	`bus_off symbol`	`Bus-off control`	Switch node off bus	`PCS -> PMA`	`aui_if.bus_off` (`std_logic`)	On `Bus_off_request` from FCE
7.4.2.3, 8.1.3.4	`bus_off_release symbol`	`Bus-off release control`	Release node from bus-off	`PCS -> PMA`	`aui_if.bus_off_release` (`std_logic`)	On `Bus_off_release_request` from FCE
7.4.3	`input symbol`	`Dominant/recessive symbol`	Indicate physical input symbol	`PMA -> PCS`	`aui_if.rx` (`std_logic`)	Continuously driven by PMA

Table 8: Interface definition for `fce_llc_if`.

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
8.1.3.2	`Normal_mode_request`	`Mode request`	Request reset to normal mode	`LLC -> FCE`	`fce_llc_if.normal_mode_request` (`boolean`)	Issued on startup/restart

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
8.1.3.2	`Normal_mode_response`	`Mode response`	Acknowledge normal-mode request	`FCE -> LLC`	`fce_llc_if.normal_mode_response` (`boolean`)	Returned after FCE processing
8.1.3.2	`Bus_off`	`Bus-off status`	Indicate node is bus-off	`FCE -> LLC`	`fce_llc_if.bus_off` (`boolean`)	Asserted on bus-off transition

Table 9: Interface definition for `fce_mac_if`.

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
8.1.3.3	`Transmit/receive`	`Transfer mode context`	Report current TX/RX context	`MAC -> FCE`	`fce_mac_if.transmitting` (`boolean`)	Updated with MAC transfer context
8.1.3.3	`Error`	`Error event`	Report detected protocol error	`MAC -> FCE`	`fce_mac_if.error` (`boolean`)	Pulse on bit/stuff/CRC/form/ACK error
8.1.3.3	`Primary_error`	`Primary error event`	Report primary error condition	`MAC -> FCE`	`fce_mac_if.primary_error` (`boolean`)	Pulse on primary error condition
8.1.3.3	`Error/overload flag`	`EF/OF state`	Report EF/OF transmission state	`MAC -> FCE`	`fce_mac_if.sending_error_flag` (`boolean`)	Asserted during EF/OF transmission
8.1.3.3	`Counters_unchanged`	`Counter-update qualifier`	Qualify counter exception path	`MAC -> FCE`	`fce_mac_if.counters_unchanged` (`boolean`)	Asserted on rule-c exception cases
8.1.3.3	`Error_delimiter_too_la`	`Late delimiter event`	Report late error-delimiter condition	`MAC -> FCE`	`fce_mac_if.error_delimiter_too_` (`boolean`)	Asserted on late delimiter condition
8.1.3.3	`Successful_transfer`	`Transfer completion event`	Report successful TX/RX completion	`MAC -> FCE`	`fce_mac_if.successful_transfer` (`boolean`)	Pulse on successful frame transfer

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
8.1.3.3	`Error_passive_response`	`State response`	Report entry into error-passive state	`MAC -> FCE`	`fce_mac_if.error_passive_respor` (`boolean`)	On state transition completion
8.1.3.3	`Error_active_response`	`State response`	Report return to error-active state	`MAC -> FCE`	`fce_mac_if.error_active_respons` (`boolean`)	On state transition completion
8.1.3.3	`Error_passive_request`	`State request`	Request MAC enter error-passive state	`FCE -> MAC`	`fce_mac_if.error_passive` (`boolean`)	On TEC/REC threshold crossing
8.1.3.3	`Error_active_request`	`State request`	Request MAC return to error-active state	`FCE -> MAC`	`fce_mac_if.error_active` (`boolean`)	On TEC/REC recovery

Table 10: Interface definition for `fce_pcs_if`.

ISO ref.	ISO symbol	ISO payload	ISO semantics	Direction	Implementation mapping	Implementation notes
8.1.3.4	`Bus_off_request`	`Bus-off request`	Request node switch-off from bus	`FCE -> PCS`	`fce_pcs_if.bus_off_request` (`boolean`)	On bus-off transition condition
8.1.3.4	`Bus_off_release_request`	`Bus-off release request`	Request node re-enable from bus-off	`FCE -> PCS`	`fce_pcs_if.bus_off_release_requ` (`boolean`)	On restart/reintegration
8.1.3.4	`Bus_off_response`	`Bus-off response`	Acknowledge bus-off request	`PCS -> FCE`	`fce_pcs_if.bus_off_response` (`boolean`)	Returned after bus-off action
8.1.3.4	`Bus_off_release_respons`	`Bus-off release response`	Acknowledge bus-off-release request	`PCS -> FCE`	`fce_pcs_if.bus_off_release_resp` (`boolean`)	Returned after release action

1.3 Protocol-Driven Type System

The shared type system encodes protocol semantics directly into the interface bundles defined in Section 1.2. This makes the protocol semantics visible in the implementation and in simulation waveforms. As shown in Figure 4, the hierarchy is rooted in ISO-derived protocol constants, from which frame layout constants, semantic enumeration types, and composite record types are derived.

1.4 LLC Sub-layer

Responsible for frame buffering and retransmission management. It provides an Avalon-ST interface to the user application and communicates with the FCE to handle retransmission limits and error status reporting.

1.4.1 `can_llc_tx`

`can_llc_tx` buffers an incoming LLC frame from the user, streams it byte-by-byte to the MAC serializer, and manages retransmission on bus disturbance up to the ISO-mandated limit (ISO ref.: 6.4.5, 6.5).

1.4.2 `can_llc_rx`

`can_llc_rx` receives a reconstructed LLC frame from the MAC layer, applies acceptance filtering, and delivers accepted frames to the user over the `llc_rx_if` Avalon-ST stream (ISO ref.: 6.4.5).

1.5 MAC Sub-layer

The core of the protocol logic. It handles bit serialization, CRC calculation, and bit stuffing. It coordinates the overall frame state machine and interacts closely with the Fault Confinement Entity (FCE) to manage error counters (TEC/REC) and node state (Error Active/Passive/Bus Off) based on protocol violations.

1.5.1 `can_mac_tx`

The MAC TX layer (Figure 5) is composed of four main components:

1. **tx_mac_ser** (Serializer): Converts LLC bytes into a serial bit stream with polarity information
2. **tx_mac_fsm** (Frame State Machine): Coordinates frame transmission, controls all submodules
3. **bit_stuffer_fd** (Bit Stuffer): Implements CAN/CAN-FD bit stuffing rules
4. **crc_fd** (CRC Engine): Generates CRC-15/CRC-17/CRC-21 based on frame type

1.5.1.1 `can_mac_tx` Internal Interfaces The interfaces between MAC sub-components are implementation-defined and carry no direct ISO service primitive mapping. Table 11, Table 12, and Table 13 define each bidirectional bundle; the Direction column identifies the driving component for each field.

Table 11: Interface definition for `can_mac_fsm_ser_tx_if`.

Field	Type	Direction	Description
`data`	`polarity_t`	`SER → FSM`	Current bit polarity; FSM reads this when `valid` is asserted
`valid`	`boolean`	`SER → FSM`	Asserted while a bit is available for the FSM to consume

Field	Type	Direction	Description
frame_params	frame_params_t	SER → FSM	Frame parameters computed from the two config bytes; valid for the lifetime of the frame
ready	boolean	FSM → SER	Pulsed when the FSM has consumed the current bit; advances the serializer to the next bit
transfer_status	transfer_status_t	FSM → SER	Frame outcome; any non-ongoing value terminates serialization

Table 12: Interface definition for can_mac_fsm_bs_tx_if.

Field	Type	Direction	Description
data	polarity_t	FSM → BS	Bit polarity fed into the bit stuffer
valid	boolean	FSM → BS	Pulsed when a new bit is presented to the stuffer
start	boolean	FSM → BS	Pulsed at frame start to reinitialize the stuffer state
data	polarity_t	BS → FSM	Polarity of the required stuff bit
valid	boolean	BS → FSM	Asserted when a stuff bit insertion is required
sbc	std_logic_vector(3:0)	BS → FSM	Gray-coded stuff bit count with parity, used in the FD fixed-stuffing region

Table 13: Interface definition for can_mac_fsm_crc_tx_if.

Field	Type	Direction	Description
crc_poly_select	std_logic_vector(1:0)	FSM → CRC	Selects the active CRC polynomial: CRC-15, CRC-17, or CRC-21
valid	boolean	FSM → CRC	Pulsed when data should be included in the CRC accumulation
data	std_logic	FSM → CRC	Bit value to feed into the CRC engine

Field	Type	Direction	Description
crc	crc_vector_t	CRC → FSM	Current CRC register value; FSM reads this when serializing the CRC field

1.5.1.2 can_mac_fsm_tx

The MAC TX FSM (Figure 6) orchestrates all frame transmission activity. It coordinates the Serializer, Bit Stuffer, and CRC engine, and drives the mac_pcs_if output bit on each sample-point strobe from the PCS.

The transmitting_frame state is the most complex. On entry, initialize_frame_transmission() resets the Serializer (can_mac_ser_tx), the Bit Stuffer (can_mac_bs_tx, MAC Sub-layer, ISO ref.: 8.5), and the CRC engine (can_mac_crc_tx, MAC Sub-layer, ISO ref.: 8.5.4). Each sample-point strobe from the PCS Sub-layer (ISO ref.: 7.2-7.4) then drives the following sequence:

1. get_observed_mac_frame_bit_info() reads the bus polarity at the sample point and compares it against the transmitted bit, returning an event record with mismatch and stuff-bit flags.
2. If an SSP error was latched from the previous bit period (CAN-FD data phase TDC, PCS Sub-layer, ISO ref.: 7.3.4), that error is processed before the current event.
3. Based on the event, either transmit_stuff_bit() or transmit_normal_bit() drives the next bit and updates the CRC and stuff-bit counter.
4. A detected error or polarity mismatch triggers a transition to transmitting_active_error_flag or transmitting_passive_error_flag depending on the fault confinement state reported by the FCE (Fault Confinement Entity, ISO ref.: 8.1.3-8.1.4).

1.5.1.3 can_mac_ser_tx

can_mac_ser_tx converts the LLC byte stream into a serial polarity bit stream for the MAC FSM. Its four-state FSM (Figure 7) manages the two-byte configuration handshake, byte fetching, and bit-by-bit serialization.

1.5.2 can_mac_rx

can_mac_rx deserializes the bit stream received from the PCS, performs bit destuffing and CRC verification, and reconstructs the LLC frame for delivery to can_llc_rx (ISO ref.: 6.6).

1.6 PCS Sub-layer

Handles bit timing and synchronization. It generates the sample point (SP) and secondary sample point (SSP) strobes. It provides bit-level monitoring data to the FCE to detect synchronization and timing errors.

1.6.1 can_pcs_tx

can_pcs_tx implements bit timing and Transmitter Delay Compensation for the TX path. It generates the sample point (SP) strobe used by the MAC FSM to advance frame state, and - when TDC is configured - a secondary sample point (SSP) strobe for data-phase bit monitoring. The FSM (Figure 8) has four states reflecting the two bit rates and the TDC measurement window. In the measuring_delay state, the module counts the physical TX-to-RX propagation delay (TDCV, ISO ref.: 7.3.4) by timing the dominant edge on the looped-back RX input relative to the TX output edge, then adds the configured TDCO offset to derive the SSP position for subsequent data-phase bits.

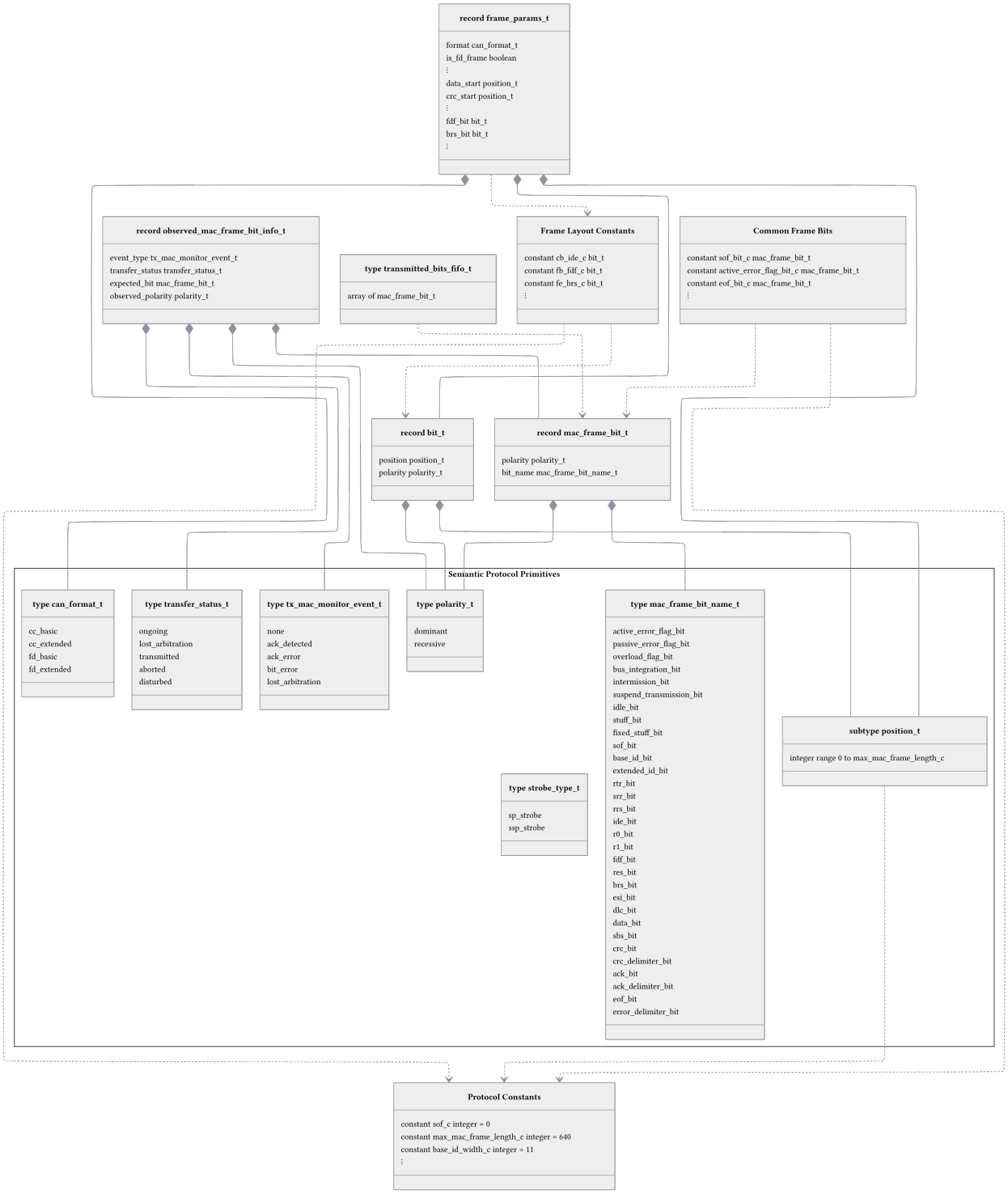

Figure 4: Type and constant hierarchy. The Semantic Protocol Primitives namespace groups the enumeration types and subtypes that comprise the protocol semantic primitives. Compound record types compose these primitives: bit_t pairs a bit position with a polarity and underpins the frame layout constants. mac_frame_bit_t carries semantic context by combining a polarity with a protocol bit name, so each transmitted bit remains identifiable throughout the design. frame_params_t aggregates format flags, field boundary positions, and format-specific control bit positions, computed once per frame (Section 1.5.1.3). observed_mac_frame_bit_info_t and transmitted_bits_fifo_t support bus monitoring and TDC-delayed bit comparison (Section 1.5.1.2). Constant groups derive from the root protocol constants.

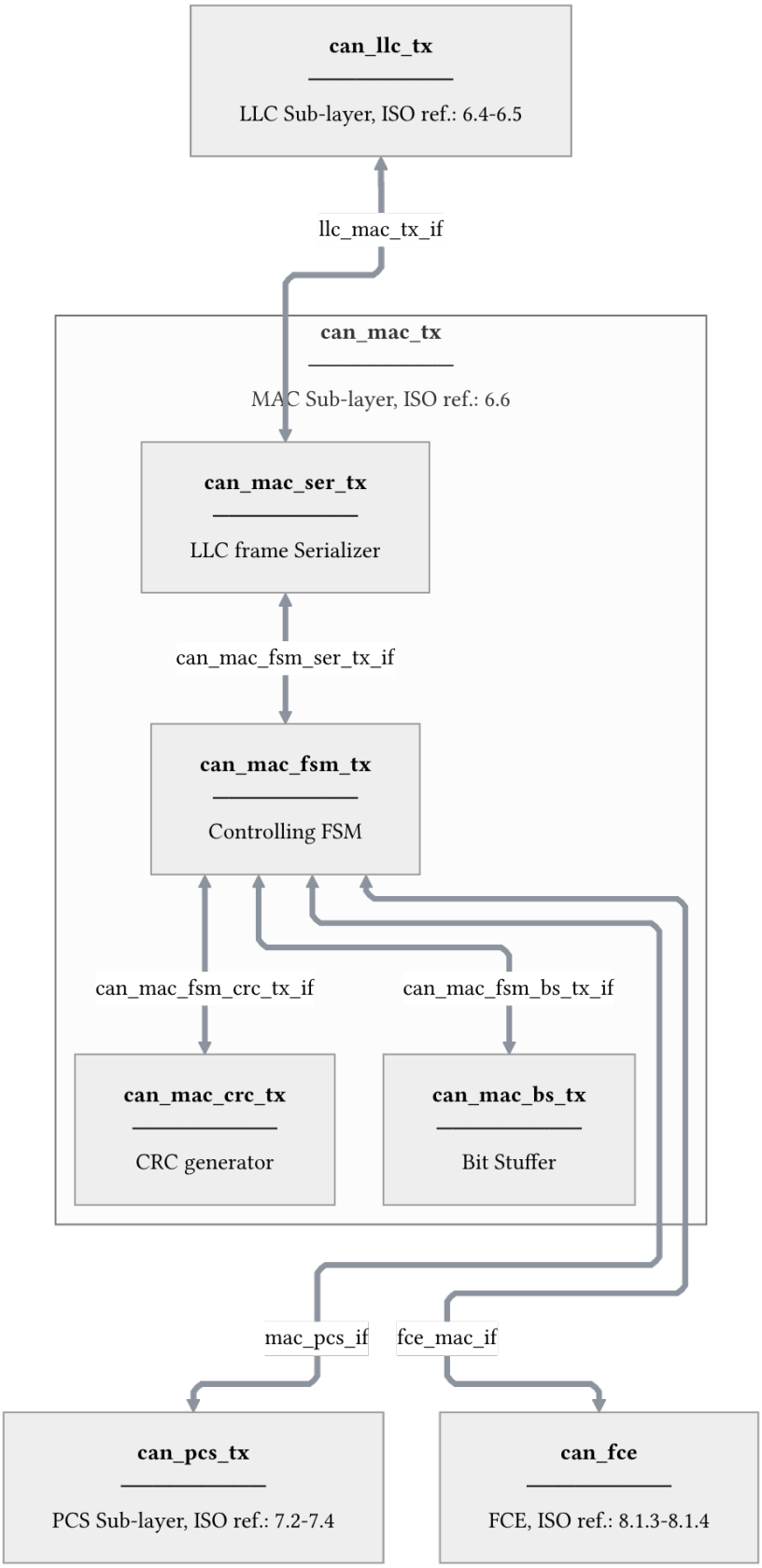

Figure 5: MAC TX layer architecture showing internal component interconnections and external interfaces (LLC, PCS, FCE). Internal interface definitions are provided for can_mac_fsm_ser_tx_if (Table 11), can_mac_fsm_bs_tx_if (Table 12), and can_mac_fsm_crc_tx_if (Table 13).

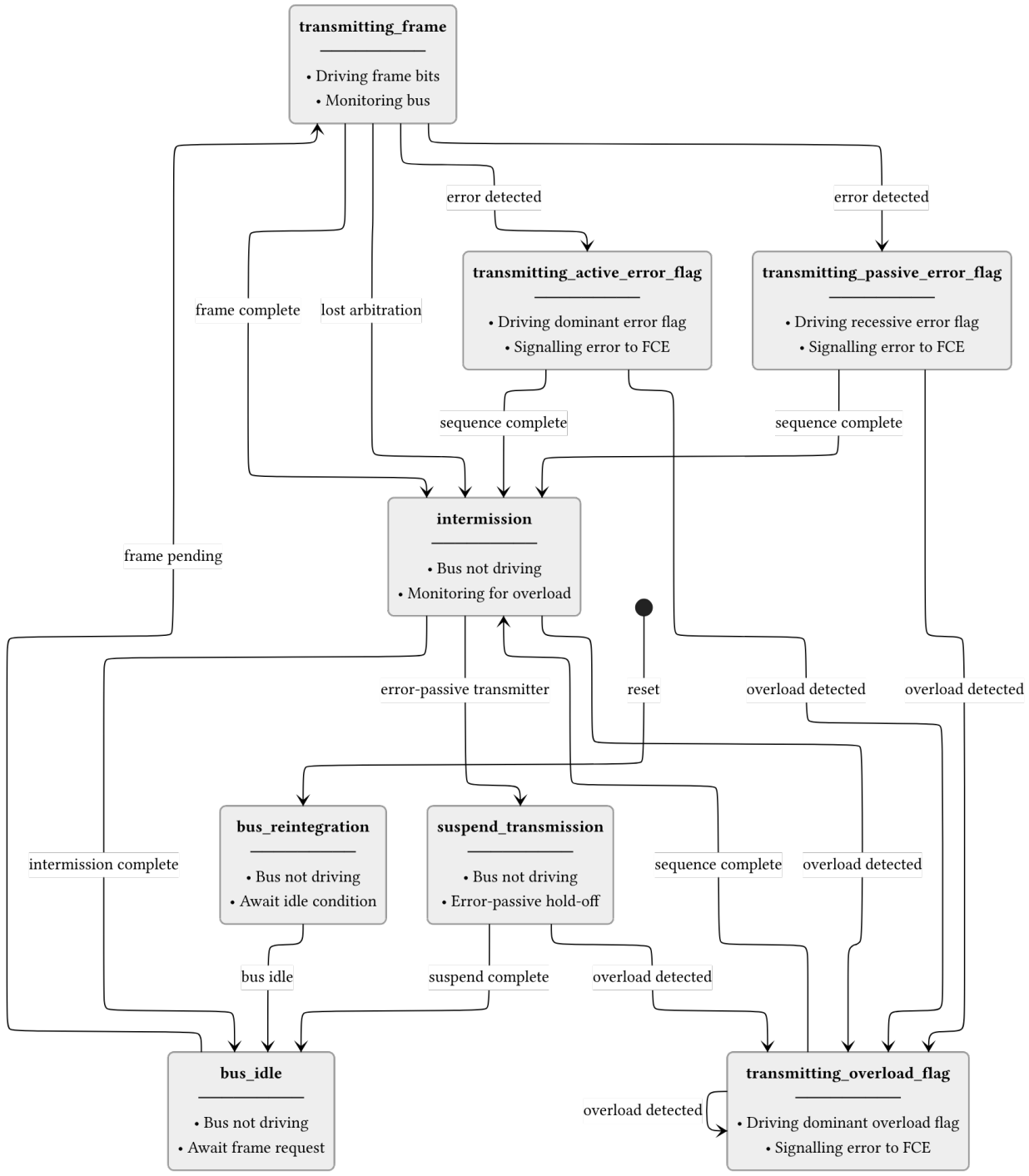

Figure 6: MAC TX FSM state transitions. Error-active and error-passive paths share the same flag+delimiter sequence but drive different polarities. Overload transitions (dominant in intermission bits 0-1, or dominant on last delimiter bit) are omitted for clarity.

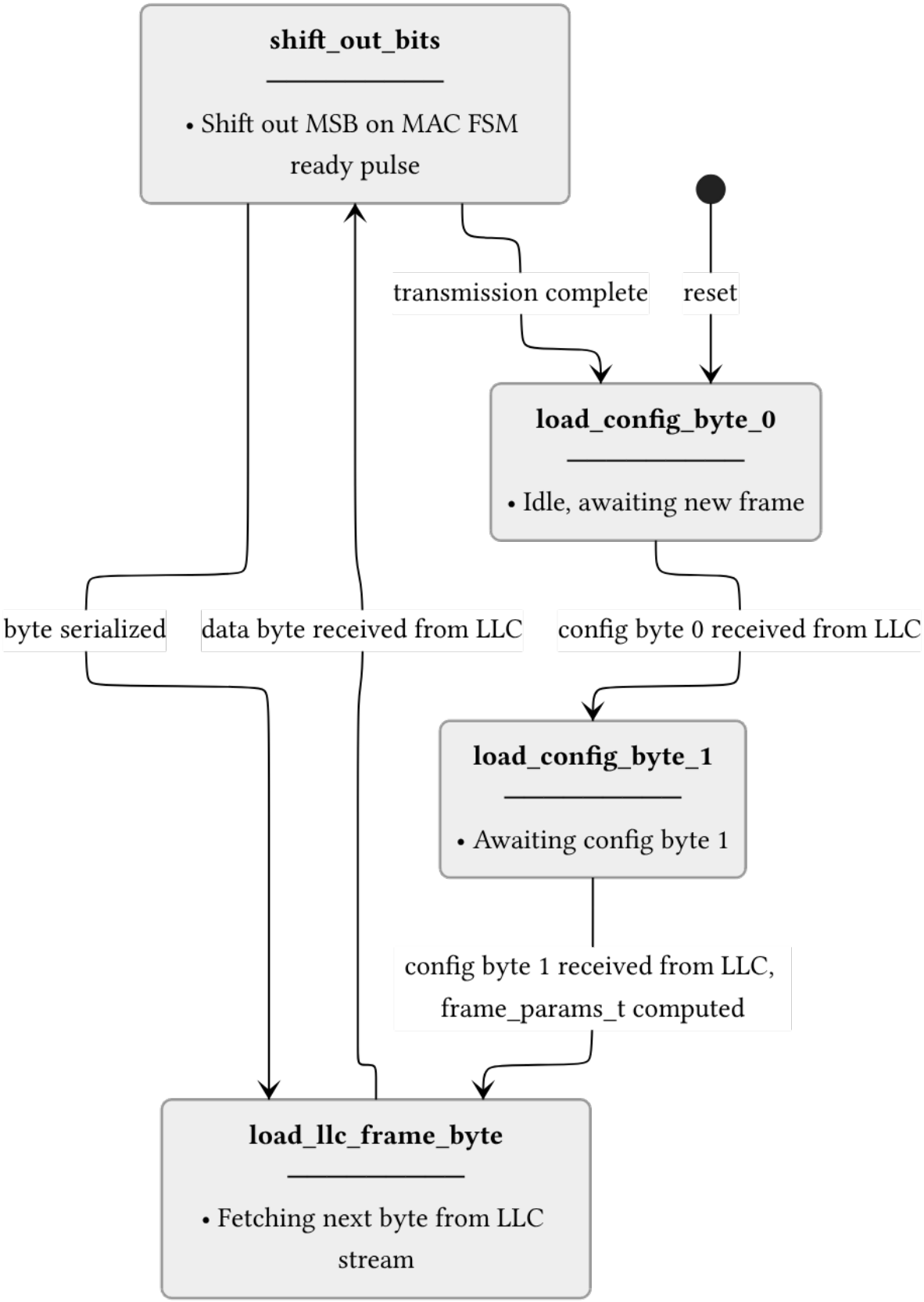

Figure 7: can_mac_ser_tx FSM. The serializer accepts LLC frame bytes over a ready/valid handshake and shifts them out MSB-first to the MAC FSM one bit per ready pulse. Frame parameters are computed once from the two config bytes and cached in frame_params_t for use by the FSM throughout the frame.

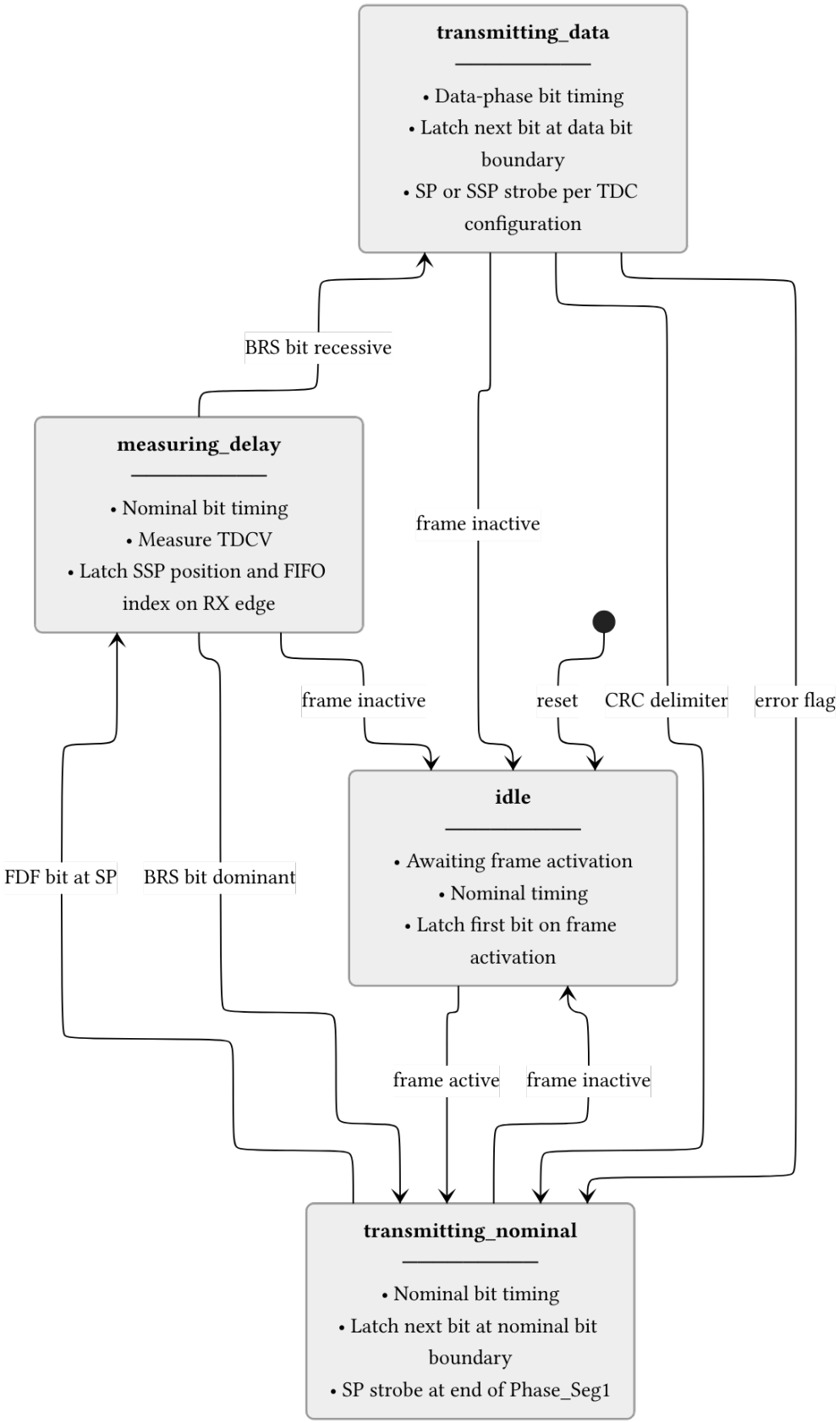

Figure 8: can_pcs_tx FSM. The measuring_delay state is entered on the FDF sample point to measure TDCV (Transmitter Delay Compensation Value, ISO ref.: 7.3.4). The BRS bit boundary determines whether data-phase timing is used. All non-idle states return to idle when the frame becomes inactive.

1.6.2 `can_pcs_rx`

`can_pcs_rx` implements bit timing and synchronization for the RX path. It performs hard and soft synchronization on the incoming bus signal and generates the sample point strobe used by the MAC RX layer to latch each received bit (ISO ref.: 7.2-7.3).
